

INDICATOR_DEVIATION – End-to-end implementation

The deviation is its own table, linked by foreign key to INDICATOR_CONFIGURATION and to FILTERING_LAW. It stores all thirteen parameters, the TE and the dates. The live configuration is never modified. Steps 1–9 are server, 10–15 client. One rebuild and redeploy at the end. Part 1 undoes the earlier DEVIATION design; skip it if you have not built that yet.

Why Apply is not saving today: the previous design wrote the deviation into ThresholdConfigurationDto and those fields were removed. Nothing writes a deviation anywhere until Step 15 is in place.

Part 1 – Undo the previous revision

If you built the DEVIATION table and its classes from the earlier revision document, do these first. If you have not started that yet, skip to Part 2.

Step 1 — Rename the three classes – use Eclipse refactoring, not find and replace

Select each class name in the editor, press Alt+Shift+R, type the new name, tick "Update references". Eclipse fixes every import and call site.

Old	New
Deviation	IndicatorDeviation
DeviationDto	IndicatorDeviationDto
DeviationDao	IndicatorDeviationDao

Rename the file as well if Eclipse does not do it automatically. The methods keep their names – findActiveDeviations, createDeviations and the rest are still correct.

Step 2 — Drop the DEVIATION table

Run this in pgAdmin before creating the new one in Part 2.

```
DROP TABLE IF EXISTS public.deviation;
```

Step 3 — IndicatorDeviation entity – replace the two id columns with a foreign key

The earlier version stored aircraftId and indicatorId as loose strings. Benoit asked for a real foreign key to indicator_configuration.

```
// DELETE
@Column(name = "AIRCRAFT_ID", nullable = false) private String aircraftId;
@Column(name = "INDICATOR_ID", nullable = false) private String indicatorId;

// ADD
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "INDICATOR_CONFIGURATION_ID", nullable = false)
private IndicatorConfiguration indicatorConfiguration;
```

Step 4 — IndicatorDeviation entity – make the filtering law a foreign key

It was a plain name string. It has to point at filtering_law.id.

```
// DELETE
@Column(name = "FILTERING_LAW") private String filteringLaw;

// ADD
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "FILTERING_LAW_ID")
private FilteringLaw filteringLaw;
```

Step 5 — IndicatorDeviation entity – replace isActive with named queries

The inline JPQL in the DAO went through aircraftId, which no longer exists on the entity. Move both queries onto the entity as named queries. The full entity, including them, is in Step 10.

```
// the @NamedQuery pair from Step 7 goes above the class declaration
```

Step 6 — IndicatorDeviationDto – replace the two id fields

Same change as the entity, but as plain strings because a DTO cannot carry a lazy entity.

```
// DELETE
private String aircraftId;
private String indicatorId;
private String filteringLaw;

// ADD
private String indicatorConfigurationId;
private String filteringLawId;
private String filteringLawName;    // display only
```

Step 7 — IndicatorDeviationDao – replace the two inline queries

Delete both createQuery calls. They filtered on d.aircraftId, which is gone. The replacements in Step 13 use named queries and add two finder methods the business layer needs to resolve the foreign keys.

```
// DELETE both of these
public List<Deviation> findActive(String aircraftId) {
    return manager.createQuery(
        "SELECT d FROM Deviation d WHERE d.aircraftId = :aircraftId "
        + "AND d.endDate IS NULL", Deviation.class)
    ...
}
public List<Deviation> findHistory(String aircraftId) { ... }
```

Step 8 — HealthCommonBusiness – the old getFilteringLawNames

Delete it from both the interface and the implementation. The client now needs the law id as well as the name, so Step 15 replaces it with getFilteringLaws returning DTOs.

```
// DELETE from IHealthCommonBusiness and HealthCommonBusiness
List<String> getFilteringLawNames(String aircraftId) throws EntityNotFoundException;
```

Keep the `getFilteringLaws().size()` line in `HealthDao.getLatestConfiguration`. Without it the lazy set still throws when iterated.

Part 2 – Server

Step 9 — INDICATOR_DEVIATION table – run in pgAdmin

Written in the same style as the indicator_configuration script: varchar(255) with the collate clause, foreign key columns last, fk_ and ix_ naming.

```

-- Table: public.indicator_deviation

-- DROP TABLE IF EXISTS public.indicator_deviation;

CREATE TABLE IF NOT EXISTS public.indicator_deviation
(
    id character varying(255) COLLATE pg_catalog."default" NOT NULL,
    te_reference character varying(255) COLLATE pg_catalog."default" NOT NULL,
    start_date timestamp without time zone NOT NULL,
    end_date timestamp without time zone,
    severity character varying(255) COLLATE pg_catalog."default",
    threshold_level character varying(255) COLLATE pg_catalog."default",
    threshold_type character varying(255) COLLATE pg_catalog."default",
    threshold_value double precision,
    min_value double precision,
    max_value double precision,
    learning_time bigint,
    work_card character varying(255) COLLATE pg_catalog."default",
    minimum_deviation double precision,
    mean_factor double precision,
    deviation_coefficient double precision,
    nb_successive_points integer,
    indicator_configuration_id character varying(255) COLLATE pg_catalog."default" NOT NULL,
    filtering_law_id character varying(255) COLLATE pg_catalog."default",
    CONSTRAINT indicator_deviation_pkey PRIMARY KEY (id),
    CONSTRAINT fk_indicator_deviation_indicator_configuration_id
        FOREIGN KEY (indicator_configuration_id)
        REFERENCES public.indicator_configuration (id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT fk_indicator_deviation_filtering_law_id
        FOREIGN KEY (filtering_law_id)
        REFERENCES public.filtering_law (id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.indicator_deviation
    OWNER to postgres;

-- Index: ix_ind_dev_ind_con_id
CREATE INDEX IF NOT EXISTS ix_ind_dev_ind_con_id
    ON public.indicator_deviation USING btree
    (indicator_configuration_id COLLATE pg_catalog."default" ASC NULLS LAST)
    TABLESPACE pg_default;

-- Index: ix_ind_dev_fil_law_id
CREATE INDEX IF NOT EXISTS ix_ind_dev_fil_law_id
    ON public.indicator_deviation USING btree
    (filtering_law_id COLLATE pg_catalog."default" ASC NULLS LAST)
    TABLESPACE pg_default;

```

Step 10 — IndicatorDeviation – the complete entity

mgs-jpa-model/src/main/java/com/eurocopter/mgs/datamodel/health/monitoring/IndicatorDeviation.java

The full reference version. If you did Part 1, the edits there have already brought your class most of the way – compare against this and fill the gaps.

```

@Entity
@Table(name = "INDICATOR_DEVIATION")
@NamedQuery(name = "findActiveDeviationsByAircraft",
    query = "SELECT d FROM AircraftConfiguration ac JOIN ac.configurations c "
        + "JOIN TREAT(c AS HealthMonitoringConfiguration).mechanicalComponentConfigurations mcc "
        + "JOIN mcc.indicatorConfigurations ic JOIN IndicatorDeviation d "
        + "ON d.indicatorConfiguration = ic "
        + "WHERE ac.aircraft.id = :aircraftId AND d.endDate IS NULL")
@NamedQuery(name = "findDeviationHistoryByAircraft",
    query = "SELECT d FROM AircraftConfiguration ac JOIN ac.configurations c "
        + "JOIN TREAT(c AS HealthMonitoringConfiguration).mechanicalComponentConfigurations mcc "
        + "JOIN mcc.indicatorConfigurations ic JOIN IndicatorDeviation d "
        + "ON d.indicatorConfiguration = ic "
        + "WHERE ac.aircraft.id = :aircraftId AND d.endDate IS NOT NULL")
public class IndicatorDeviation implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @Column(name = "ID", nullable = false)
    private String id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "INDICATOR_CONFIGURATION_ID", nullable = false)
    private IndicatorConfiguration indicatorConfiguration;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "FILTERING_LAW_ID")
    private FilteringLaw filteringLaw;

    @Column(name = "TE_REFERENCE", nullable = false)
    private String teReference;

    @Temporal(TemporalType.TIMESTAMP)
    @Column(name = "START_DATE", nullable = false)
    private Date startDate;

    @Temporal(TemporalType.TIMESTAMP)
    @Column(name = "END_DATE")
    private Date endDate;

    @Column(name = "SEVERITY")           private String severity;
    @Column(name = "THRESHOLD_LEVEL")   private String thresholdLevel;
    @Column(name = "THRESHOLD_TYPE")    private String thresholdType;
    @Column(name = "THRESHOLD_VALUE")   private Double thresholdValue;
    @Column(name = "MIN_VALUE")         private Double minValue;
    @Column(name = "MAX_VALUE")         private Double maxValue;
    @Column(name = "LEARNING_TIME")     private Long learningTime;
    @Column(name = "WORK_CARD")         private String workcard;
    @Column(name = "MINIMUM_DEVIATION") private Double minimumDeviation;
    @Column(name = "MEAN_FACTOR")       private Double meanFactor;
    @Column(name = "DEVIATION_COEFFICIENT") private Double deviationCoefficient;
    @Column(name = "NB_SUCCESSIVE_POINTS") private Integer nbSuccessivePoints;

    IndicatorDeviation() {
    }

    public IndicatorDeviation(String id) {
        this.id = id;
    }

    public boolean isActive() {
        return endDate == null;
    }

    // plus getters and setters for every field
}

```

Imports: `java.io.Serializable`, `java.util.Date`, and from `jakarta.persistence` – `Entity`, `Table`, `Id`, `Column`, `ManyToOne`, `JoinColumn`, `FetchType`, `Temporal`, `TemporalType`, `NamedQuery`. `TREAT` is needed in the query because `AircraftConfiguration.configurations` is a `Map` of the `Configuration` base class, and only the `HealthMonitoringConfiguration` subclass has `mechanicalComponentConfigurations`.

Step 11 — FilteringLawDto – new

`mgs-services-interfaces/src/main/java/com/eurocopter/mgs/service/health/common/dto/FilteringLawDt`

o.java

The client combo shows names but must save the id, so both cross the wire.

```
public class FilteringLawDto implements Serializable {
    private static final long serialVersionUID = 1L;
    private String id;
    private String name;
    // getters and setters
}
```

Step 12 — IndicatorDeviationDto – new

mgs-services-interfaces/src/main/java/com/eurocopter/mgs/service/health/common/dto/IndicatorDeviationDto.java

Plain fields only. The foreign keys travel as String ids so no lazy proxy reaches the client.

```
public class IndicatorDeviationDto implements Serializable {
    private static final long serialVersionUID = 1L;

    private String id;
    private String indicatorConfigurationId;
    private String filteringLawId;
    private String filteringLawName;    // display only
    private String teReference;
    private Date startDate;
    private Date endDate;

    private String severity;
    private String thresholdLevel;
    private String thresholdType;
    private Double thresholdValue;
    private Double minValue;
    private Double maxValue;
    private Long learningTime;
    private String workcard;
    private Double minimumDeviation;
    private Double meanFactor;
    private Double deviationCoefficient;
    private Integer nbSuccessivePoints;
    // getters and setters
}
```

Step 13 — IndicatorDeviationDao – new

mgs-services-impl/src/main/java/com/eurocopter/mgs/service/health/common/ejb/IndicatorDeviationDao.java

Same package and same @Local @Stateless pattern as HealthDao.

```

package com.eurocopter.mgs.service.health.common.ejb;

@Local
@Stateless
public class IndicatorDeviationDao {

    @PersistenceContext
    private EntityManager manager;

    public void create(IndicatorDeviation deviation) {
        manager.persist(deviation);
    }

    public void acknowledge(String deviationId, Date endDate) {
        IndicatorDeviation deviation = manager.find(IndicatorDeviation.class, deviationId);
        if (deviation != null) {
            deviation.setEndDate(endDate);
        }
    }

    public List<IndicatorDeviation> findActive(String aircraftId) {
        TypedQuery<IndicatorDeviation> q =
            manager.createNamedQuery("findActiveDeviationsByAircraft", IndicatorDeviation.class);
        q.setParameter("aircraftId", aircraftId);
        return q.getResultList();
    }

    public List<IndicatorDeviation> findHistory(String aircraftId) {
        TypedQuery<IndicatorDeviation> q =
            manager.createNamedQuery("findDeviationHistoryByAircraft", IndicatorDeviation.class);
        q.setParameter("aircraftId", aircraftId);
        return q.getResultList();
    }

    public IndicatorConfiguration findIndicatorConfiguration(String id) {
        return manager.find(IndicatorConfiguration.class, id);
    }

    public FilteringLaw findFilteringLaw(String id) {
        return id == null ? null : manager.find(FilteringLaw.class, id);
    }
}

```

Step 14 — IHealthCommonService and HealthCommonService

Add the six methods to the interface and six one-line delegates to the EJB, with the DAO injected the same way HealthDao already is.

```

// field on HealthCommonService
@EJB
private IndicatorDeviationDao deviationDao;

// interface + implementation
List<IndicatorDeviation> findActiveDeviations(String aircraftId);
List<IndicatorDeviation> findDeviationHistory(String aircraftId);
void createDeviation(IndicatorDeviation deviation);
void acknowledgeDeviation(String deviationId, Date endDate);
IndicatorConfiguration findIndicatorConfiguration(String id);
FilteringLaw findFilteringLaw(String id);

```

Step 15 — IHealthCommonBusiness and HealthCommonBusiness – replace getFilteringLawNames

The client now needs the law id as well as its name, so this returns DTOs. Delete the old getFilteringLawNames from both the interface and the implementation.

```

@Override
public List<FilteringLawDto> getFilteringLaws(String aircraftId) throws EntityNotFoundException {
    List<FilteringLawDto> result = new ArrayList<>();
    HealthMonitoringConfiguration configuration = getLatestConfiguration(aircraftId);
    for (FilteringLaw law : configuration.getFilteringLaws()) {
        FilteringLawDto dto = new FilteringLawDto();
        dto.setId(law.getId());
        dto.setName(law.getName());
        result.add(dto);
    }
    return result;
}

```

The `getFilteringLaws().size()` line added earlier to `HealthDao.getLatestConfiguration` stays. Without it the lazy set still throws when iterated.

Step 16 — HealthCommonBusiness – the four deviation methods

Entity to DTO conversion happens here, server side, so the lazy foreign keys resolve while the session is open.

```

@Override
public List<IndicatorDeviationDto> findActiveDeviations(String aircraftId) {
    return toDtos(healthCommonService.findActiveDeviations(aircraftId));
}

@Override
public List<IndicatorDeviationDto> findDeviationHistory(String aircraftId) {
    return toDtos(healthCommonService.findDeviationHistory(aircraftId));
}

@Override
public void createDeviations(List<IndicatorDeviationDto> dtos) {
    for (IndicatorDeviationDto dto : dtos) {
        IndicatorDeviation d = new IndicatorDeviation(UUID.randomUUID().toString());
        d.setIndicatorConfiguration(
            healthCommonService.findIndicatorConfiguration(dto.getIndicatorConfigurationId()));
        d.setFilteringLaw(healthCommonService.findFilteringLaw(dto.getFilteringLawId()));
        d.setTeReference(dto.getTeReference());
        d.setStartDate(dto.getStartDate());
        d.setEndDate(null);
        d.setSeverity(dto.getSeverity());
        d.setThresholdLevel(dto.getThresholdLevel());
        d.setThresholdType(dto.getThresholdType());
        d.setThresholdValue(dto.getThresholdValue());
        d.setMinValue(dto.getMinValue());
        d.setMaxValue(dto.getMaxValue());
        d.setLearningTime(dto.getLearningTime());
        d.setWorkcard(dto.getWorkcard());
        d.setMinimumDeviation(dto.getMinimumDeviation());
        d.setMeanFactor(dto.getMeanFactor());
        d.setDeviationCoefficient(dto.getDeviationCoefficient());
        d.setNbSuccessivePoints(dto.getNbSuccessivePoints());
        healthCommonService.createDeviation(d);
    }
}

@Override
public void acknowledgeDeviations(List<String> deviationIds) {
    Date now = new Date();
    for (String id : deviationIds) {
        healthCommonService.acknowledgeDeviation(id, now);
    }
}

private List<IndicatorDeviationDto> toDtos(List<IndicatorDeviation> deviations) {
    List<IndicatorDeviationDto> result = new ArrayList<>();
    for (IndicatorDeviation d : deviations) {
        IndicatorDeviationDto dto = new IndicatorDeviationDto();
        dto.setId(d.getId());
        dto.setIndicatorConfigurationId(d.getIndicatorConfiguration().getId());
        if (d.getFilteringLaw() != null) {
            dto.setFilteringLawId(d.getFilteringLaw().getId());
            dto.setFilteringLawName(d.getFilteringLaw().getName());
        }
        dto.setTeReference(d.getTeReference());
        dto.setStartDate(d.getStartDate());
        dto.setEndDate(d.getEndDate());
        dto.setSeverity(d.getSeverity());
        dto.setThresholdLevel(d.getThresholdLevel());
        dto.setThresholdType(d.getThresholdType());
        dto.setThresholdValue(d.getThresholdValue());
        dto.setMinValue(d.getMinValue());
        dto.setMaxValue(d.getMaxValue());
        dto.setLearningTime(d.getLearningTime());
        dto.setWorkcard(d.getWorkcard());
        dto.setMinimumDeviation(d.getMinimumDeviation());
        dto.setMeanFactor(d.getMeanFactor());
        dto.setDeviationCoefficient(d.getDeviationCoefficient());
        dto.setNbSuccessivePoints(d.getNbSuccessivePoints());
        result.add(dto);
    }
    return result;
}

```

Step 17 — persistence.xml

Add the new entity if the persistence unit lists classes explicitly rather than scanning the package. Check how ThresholdConfiguration appears there and follow it.

```
<class>com.eurocopter.mgs.datamodel.health.monitoring.IndicatorDeviation</class>
```

Part 3 – Client

Step 18 — WThresholdModel – three more fields

com.eurocopter.mgs.business.health.model.WThresholdModel

deviationId tells a saved deviation from a new one; the other two are what the save needs and the model does not carry today.

```
private String deviationId;           // null until saved
private String filteringLawId;        // FK, alongside the existing name
private String indicatorConfigurationId; // FK to the owning configuration row
// plus getters and setters
```

Step 19 — HealthAlarmsPreferenceStore → replace getFilteringLawNames

mgs-health-app/.../healthalarms/store/HealthAlarmsPreferenceStore.java

The combo keeps showing names; the id is looked up at save time from a map held on the store.

```
private final Map<String, String> lawNameToId = new HashMap<>();

public List<String> getFilteringLawNames(Aircraft aircraft) {
    List<String> names = new ArrayList<>();
    lawNameToId.clear();
    try {
        for (FilteringLawDto dto : healthBusiness.getFilteringLaws(aircraft.getId())) {
            names.add(dto.getName());
            lawNameToId.put(dto.getName(), dto.getId());
        }
    } catch (Exception e) {
        LOGGER.warn("Could not load filtering laws", e);
    }
    names.sort(String::compareTo);
    return names;
}

public String getFilteringLawId(String name) {
    return lawNameToId.get(name);
}
```

Step 20 — HealthAlarmsPreferenceStore → createAllMechanicalComponents

Fetch the active deviations once for the aircraft, before the component loop, and pass the map down as a fourth parameter.

```
Map<String, IndicatorDeviationDto> deviationMap = new HashMap<>();
try {
    for (IndicatorDeviationDto dto : healthBusiness.findActiveDeviations(aircraft.getId())) {
        deviationMap.put(dto.getIndicatorConfigurationId() + "|" + dto.getSeverity(), dto);
    }
} catch (Exception e) {
    LOGGER.warn("Could not load active deviations", e);
}

// then
List<WThresholdModel> thresholdModels = getThresholdModels(healthMechanicalComponent,
    indicatorModels, entry.getValue().getIndicatorConfigurations(), deviationMap);
```

The key is the indicator configuration id and the severity joined with a bar, because each indicator has a RED row and an AMBER row and each can carry its own deviation with its own TE.

Step 21 — HealthAlarmsPreferenceStore → getThresholdModels

Add the fourth parameter, then extend the loop that already sets the law and the baseline.

```

private List<WThresholdModel> getThresholdModels(
    HealthMechanicalComponent healthMechanicalComponent,
    Map<Indicator, IndicatorModel> indicatorModels,
    Map<Indicator, IndicatorConfiguration> indicatorConfigurations,
    Map<String, IndicatorDeviationDto> deviationMap) {

    ...

    IndicatorConfiguration indicatorConfiguration = indicatorConfigurations.get(indicator);
    String law = indicatorConfiguration != null ? indicatorConfiguration.getFilteringLaw() : null;
    String icId = indicatorConfiguration != null ? indicatorConfiguration.getId() : null;

    for (WThresholdModel thresholdModel : indicatorThresholdsMap.values()) {

        // 1. law and baseline, from the CONFIGURATION
        thresholdModel.setFilteringLawName(law);
        thresholdModel.setIndicatorConfigurationId(icId);
        thresholdModel.setBaseline(new WThresholdBaseline(
            thresholdModel.getDefaultValue(), thresholdModel.getMinValue(), law));

        // 2. paint the stored deviation over the top, if there is one
        IndicatorDeviationDto deviation = deviationMap.get(
            icId + "|" + thresholdModel.getSeverity().name());

        if (deviation != null) {
            thresholdModel.setDeviated(true);
            thresholdModel.setDeviationId(deviation.getId());
            thresholdModel.setTeReference(deviation.getTeReference());
            thresholdModel.setDeviationStartDate(deviation.getStartDate());
            thresholdModel.setDeviationEndDate(deviation.getEndDate());
            thresholdModel.setLevel(ThresholdLevel.valueOf(deviation.getThresholdLevel()));
            thresholdModel.setType(ThresholdType.valueOf(deviation.getThresholdType()));
            thresholdModel.setDefaultValue(deviation.getThresholdValue());
            thresholdModel.setMinValue(deviation.getMinValue());
            thresholdModel.setMaxValue(deviation.getMaxValue());
            thresholdModel.setLearningTime(deviation.getLearningTime());
            thresholdModel.setWorkCardReference(deviation.getWorkcard());
            thresholdModel.setMinimumDeviation(deviation.getMinimumDeviation());
            thresholdModel.setMeanFactor(deviation.getMeanFactor());
            thresholdModel.setDeviationCoefficient(deviation.getDeviationCoefficient());
            thresholdModel.setNbSuccessivePoints(deviation.getNbSuccessivePoints());
            thresholdModel.setFilteringLawName(deviation.getFilteringLawName());
            thresholdModel.setFilteringLawId(deviation.getFilteringLawId());
        }
    }
}

```

The order of those two blocks is the whole trick. The baseline is built first, from the configuration, and the deviated values are written on top afterwards. So the baseline holds what the configuration says and the model holds what the user sees. That gap is what `isModifiedAgainstBaseline` compares and what `acknowledge` reverts to. Build the baseline after the overwrite and it would hold the deviated values, so nothing would ever register as changed again.

Step 22 — HealthAlarmsPreferenceStore – new method `saveDeviations()`

`isValidData` has already marked every changed row deviated and rejected an empty TE. This collects those rows and sends them. A row that already has a `deviationId` came from the database and must not be created again.

```

public void saveDeviations() {
    List<IndicatorDeviationDto> newDeviations = new ArrayList<>();
    Date now = new Date();

    for (Entry<Aircraft, WCMechanicalComponentModel> entry : data.entrySet()) {
        Multimap<Indicator, WCThresholdModel> thresholds = getComponentThresholds(entry.getValue());
        for (WCThresholdModel model : thresholds.values()) {
            if (!model.isDeviated() || model.getDeviationId() != null) {
                continue;
            }
            newDeviations.add(toDeviationDto(model, now));
        }
    }

    if (!newDeviations.isEmpty()) {
        try {
            healthBusiness.createDeviations(newDeviations);
        } catch (Exception e) {
            LOGGER.error("Could not save deviations", e);
        }
    }
}

private IndicatorDeviationDto toDeviationDto(WCThresholdModel model, Date now) {
    IndicatorDeviationDto dto = new IndicatorDeviationDto();
    dto.setIndicatorConfigurationId(model.getIndicatorConfigurationId());
    dto.setFilteringLawId(getFilteringLawId(model.getFilteringLawName()));
    dto.setTeReference(model.getTeReference());
    dto.setStartDate(now);

    dto.setSeverity(model.getSeverity().name()); // 1
    dto.setThresholdLevel(model.getLevel().name()); // 2
    dto.setThresholdType(model.getType().name()); // 3
    dto.setThresholdValue(model.getDefaultValue()); // 4
    dto.setMinValue(model.getMinValue()); // 5
    dto.setMaxValue(model.getMaxValue()); // 6
    dto.setLearningTime(model.getLearningTime()); // 7
    dto.setWorkcard(model.getWorkCardReference()); // 8
    dto.setMinimumDeviation(model.getMinimumDeviation()); // 9
    dto.setMeanFactor(model.getMeanFactor()); // 10
    dto.setDeviationCoefficient(model.getDeviationCoefficient()); // 11
    dto.setNbSuccessivePoints(model.getNbSuccessivePoints()); // 12
    // 13 is the filtering law, set above as its foreign key id

    return dto;
}

```

Step 23 — HealthAlarmsComposite → performApply

One line. The configuration save is untouched; the deviation save sits beside it.

```

public void performApply() {
    healthAlarmsPreferenceStore.innerSave();
    healthAlarmsPreferenceStore.saveDeviations(); // <<< add this line
    try {
        MgsPreferenceStoreManager.saveAircraftConfigurations();
        MgsPreferenceStoreManager.clearAircraftConfigurations();
    } catch (Exception e) {
        LOGGER.error("Error while saving health preferences", e);
    }
}

```

Nothing changes inside `isValidData`. What is already there – marking a row deviated when it differs from the baseline, and rejecting an empty TE – is all it does. It validates; it never writes.

Build and test

#	Action
1	Rebuild in order: mgs-jpa-model, mgs-services-interfaces, mgs-services-impl, mgs-health-business, mgs-delegate, then the client
2	Redeploy the EAR
3	Open Health, pick an aircraft, edit a Value on a row that already shows a value

#	Action
4	Pick a filtering law and type a TE reference
5	Press Apply
6	SELECT * FROM public.indicator_deviation; – expect one row with your TE, start_date set and end_date null
7	Close and reopen the Health page – the row comes back green, locked, with the TE and the law showing
8	Press Apply with an empty TE – rejected, and the row stays editable

If step 6 returns nothing, put a breakpoint on the first line of saveDeviations and check two things: that data is not empty, and that isDeviated() is true on the row you edited. If data is empty, isValidData never ran, which means the preference page did not call it.

Not yet covered

Acknowledge and delete (REQ 7 to 9), the Deviation List and History tab contents (REQ 5, 6, 11), the Manage Filtering Laws dialog, applicability by session data date (REQ 12), configuration update warnings (REQ 10), the relearn trigger, the purge job and the DACOM extraction (REQ 13). After these fifteen steps a deviation is created, stored and displayed, but it does not yet affect monitoring.